

Engineering Review & Reflection

AI Security Automation Platform — THE ARZENS Week 10 Advanced Capstone

Arooba Afghan

What worked well in the architecture

- Decoupling the five modules through a file-based JSONL queue meant each module could be built, run, and unit-tested completely independently (see tests/) before wiring the orchestrator together — this caught bugs in isolation instead of debugging five modules at once.
- Centralizing every threshold, path, and credential name in one config.yaml made it trivial to tune SOAR's confidence thresholds and re-run the pipeline without touching code.
- Building graceful degradation into the Threat Intel module (falling back to a deterministic simulated score when an API key is missing) meant the platform stayed demoable and testable without depending on live, rate-limited external services.
- Reusing the actual Week 05 Isolation Forest model + scaler (rather than retraining) kept the ML module honest to real prior work and confirmed the six-feature schema (dur, spkts, dpkts, sbytes, dbytes, rate) flows cleanly from ingestion through to detection.

What I would do differently

- Replace the JSONL file queue with a real message broker (Kafka or Redis Streams) — the current design works well for a single-machine demo but doesn't support multiple ingestion workers writing concurrently.
- Add schema validation (e.g. pydantic models) at each hand-off point, so a malformed record fails loudly at the module boundary instead of silently propagating a missing field downstream.
- Give the SOAR engine a feedback loop: when an analyst resolves a medium-confidence case, that outcome should feed back into future confidence scoring instead of every run starting from the same static weights.
- Add integration tests (not just unit tests) that run the full pipeline against a fixed sample file and assert on the final case counts, to catch regressions across module boundaries.

Performance

On the included 300-record sample log (data/raw_logs/sample_traffic_logs.csv), a full pipeline run (ingest → enrich → detect → SOAR) completes in roughly 2.3 seconds end-to-end, or about 130 events/sec, running entirely with simulated threat-intel lookups (no live API calls). Of that time, the vast majority is threat-intel enrichment — even simulated lookups iterate every record — while ML inference (batched through IsolationForest.decision_function) and SOAR routing are effectively instantaneous by comparison. With live TI API calls enabled, throughput would drop substantially and become network-bound rather than CPU-bound, which is exactly why the TI cache (24-hour TTL) exists — repeated indicators across records are only looked up once.

Scalability: handling 10x more data

- Ingestion: already streaming-capable (ingest_stream) — a 10x log volume just means more workers reading from more file/API partitions in parallel.
- Threat Intel: the TI cache is the biggest lever here — at 10x volume, indicator reuse (the same malicious domain hit by many records) means cache hit rate rises, not just raw lookup count; moving the cache to Redis would let multiple enrichment workers share it.

- ML Detection: IsolationForest inference is already vectorized and cheap; batching more records per `decision_function()` call scales near-linearly.
- SOAR: the JSON case file would become a write bottleneck under concurrent writers well before 10x — this is the first component I'd migrate to a real database.
- Dashboard: Streamlit's `cache_data` TTL already prevents re-reading the full case/queue files on every browser interaction; at 10x scale I'd paginate the case list rather than loading the full history into the dataframe.

Security: vulnerabilities in this platform

- The SMTP notification path, if enabled, sends credentials read from environment variables in plaintext over STARTTLS — acceptable for a class demo, but a production version should use an app-specific token with least-privilege scope, not a full account password.
- The case store and TI cache are unauthenticated local JSON files — anyone with filesystem access can read or tamper with case status. A production deployment needs access control and audit logging on these stores.
- Indicators (domains/IPs) are sent to third-party TI APIs (VirusTotal, AbuseIPDB, OTX) as part of enrichment — in a real deployment against real internal traffic, this is a data-exfiltration consideration that would need a review of what's permitted to leave the network.
- There's no rate limiting or input size validation on the ingestion side yet — a malformed or adversarially large log file could be used to exhaust memory before normalization catches it.
- The automated high-confidence playbook currently takes action (auto-contain) with no rollback/approval step — a false positive at the 'high' threshold would auto-block a legitimate host until manually reversed, which is a real risk in any automated SOAR system and worth a stronger review before production use.